

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №5
по курсу «Параллельные и распределенные вычисления»

Сортировка чисел на GPU. Свертка, сканирование, гистограмма.

Выполнил: М. Д. Иванченко
Группа: М8О-310Б-23
Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2026

Условие

Цель работы. Ознакомление с фундаментальными алгоритмами GPU: свертка (reduce), сканирование (blelloch scan) и гистограмма (histogram). Реализация одной из сортировок на CUDA. Использование разделяемой и других видов памяти. Исследование производительности программы с помощью утилиты nvprof (обязательно отразить в отчете).

Вариант 4. Сортировка чет-нечет.

Требуется реализовать блочную сортировку чет-нечет для чисел типа int.

Должны быть реализованы:

- Алгоритм чет-нечет сортировки для предварительной сортировки блоков.
- Алгоритм битонического слияния, с использованием разделяемой памяти.

Ограничения: $n \leq 16 * 10^6$.

Программное и аппаратное обеспечение

Графический процессор:

Характеристика	Значение
Compute Capability	7.5
Графическая память (Video Memory)	15 ГБ (15360 МБ) GDDR6
Шина памяти (Memory Interface)	256-бит
Количество мультипроцессоров (SM)	40
Количество ядер CUDA	2560
Количество тензорных ядер (Tensor Cores)	320 (3-го поколения)
Количество RT-ядер (Ray Tracing Cores)	40 (1-го поколения)
Разделяемая память (Shared Memory)	64 КБ
Константная память (Constant Memory)	64 КБ
Регистров на блок (Registers per Block)	65536
Макс. количество нитей (Max Threads)	1024
Макс. количество блоков (Max Blocks)	$2^{31}-1$

Процессор:

Параметр	Значение
Модель	Intel(R) Xeon(R) CPU @ 2.00GHz
Всего ядер	1
Всего потоков	2
Базовая частота	2.00 ГГц
Макс. частота Turbo	2.00 ГГц
Кэш L3 (Smart Cache)	40.4 МБ
Кэш L2	1.0 МБ
Кэш L1	32.8 КБ
Техпроцесс	22 нм
Виртуализация	Включена (среда виртуальной машины)

Оперативная память: 12 ГБ.

Объем жёсткого диска: 112 ГБ.

OS: Linux 6.6.122.

IDE: Google Colab.

Compiler: NVIDIA CUDA Compiler (V12.8.93).

Метод решения

В соответствии заданием была реализована блочная сортировка чёт-нечет (odd-even transposition sort) с последующим битоническим слиянием. Основная идея – разбить исходный массив на блоки, которые помещаются в разделяемую память (размер блока 1024 элемента), отсортировать каждый блок с помощью чёт-нечет сортировки, а затем выполнить глобальное битоническое слияние. Такой подход позволяет эффективно использовать быструю разделяемую память на этапе предварительной сортировки и масштабироваться на большие массивы благодаря битонической сети, которая хорошо распараллеливается на GPU. Для упрощения работы алгоритма массив дополняется до степени двойки значениями INF (максимальное целое число).

Описание программы

Программа реализована в одном файле kernel.cu, который содержит три пользовательских ядра CUDA, вспомогательную функцию swapDir и основную функцию main. Используются следующие константы: BLOCK = 256 – количество

потоков в блоке для глобальных этапов, $LOCAL = 1024$ – размер локального блока (равен количеству потоков в блоке на первом этапе), $INF = 0x7fffffff$ – значение-заполнитель для элементов вне границ массива. Данные всех этапов – целые числа типа `int`. Ввод и вывод осуществляется в бинарном формате через `stdin/stdout`, сначала читается длина массива n , затем сами числа.

Ядро `oddEvenShared` выполняет чёт-нечет сортировку одного блока данных, целиком размещённого в разделяемой памяти. Каждый блок имеет $LOCAL$ потоков, которые копируют свои элементы из глобальной памяти в разделяемый массив s . Затем в цикле по фазе выполняется сравнение соседних элементов: на чётных фазах сравниваются пары $(0,1), (2,3), \dots$, на нечётных – $(1,2), (3,4), \dots$. Для каждого сравнения вычисляется минимум и максимум, которые записываются обратно в разделяемую память. После окончания сортировки, если номер блока нечётный, производится инвертирование порядка элементов в блоке (реверс) – это необходимо для корректного последующего битонического слияния. В конце каждый поток сохраняет свой элемент обратно в глобальную память.

Ядро `bitonicGlobalShared` выполняет один шаг битонического слияния, используя разделяемую память для ускорения обменов. Блок потоков ($BLOCK \ll 1 = 512$ потоков) загружает 512 элементов из глобальной памяти в разделяемый массив. Затем для каждого потока вычисляется парный индекс $ix_j = tid \wedge j$, где j – текущий шаг слияния (степень двойки). Если ix_j находится в пределах блока и больше tid , выполняется условный обмен: направление сортировки (dir) определяется проверкой бита k в глобальном индексе. Обмен реализован через `swapDir`. Результаты записываются обратно в глобальную память. Данное ядро применяется, когда шаг j не превышает $BLOCK$ (т.е. 256), что гарантирует помещение всех необходимых данных в разделяемую память.

Ядро `bitonicGlobal` аналогично выполняет шаг битонического слияния, но работает напрямую с глобальной памятью без использования разделяемой памяти. Каждый поток обрабатывает одну пару элементов по индексам gid и $gid \wedge j$. Такое ядро используется для крупных шагов $j > BLOCK$, когда данные не умещаются в разделяемую память одного блока.

Функция `main` сначала считывает входные данные, дополняет массив до степени двойки значениями INF , копирует его на устройство. Затем запускается ядро `oddEvenShared` для предварительной сортировки каждого блока размером 1024 элемента. После этого следуют два вложенных цикла по k (размер сливаемых подмассивов, от 2048 до n_row2) и по j (текущий шаг внутри слияния). В зависимости от значения j выбирается подходящее ядро – `bitonicGlobalShared` (если $j \leq BLOCK$) или `bitonicGlobal`. После завершения всех этапов отсортированный массив (без INF) копируется обратно на хост и выводится в `stdout`.

Результаты

Время GPU:

Размер входных данных	Конфигурация	Время GPU (миллисекунды, мс)
2 048	<<1, 32>>	0.45
2 048	<<8, 256>>	0.32
2 048	<<2, 1024>>	0.30
4 096	<<1, 32>>	0.85
4 096	<<16, 256>>	0.60
4 096	<<4, 1024>>	0.55
1 048 576 (2 ²⁰)	<<4096, 256>>	15.0
1 048 576	<<1024, 1024>>	12.4
1 048 576	<<2048, 512>>	13.2
4 194 304 (2 ²²)	<<16384, 256>>	48.0
4 194 304	<<4096, 1024>>	38.7
4 194 304	<<8192, 512>>	42.0
8 388 608 (2 ²³)	<<32768, 256>>	85.0
8 388 608	<<8192, 1024>>	68.2
8 388 608	<<16384, 512>>	74.0
16 777 216 (2 ²⁴)	<<65536, 256>>	175.0
16 777 216	<<16384, 1024>>	142.0
16 777 216	<<32768, 512>>	155.0

Время CPU:

Размер массива (n)	Время CPU (мс)
2 048	0,10
4 096	0,20
1 048 576	380
4 194 304	1 520
8 388 608	3 100
16 777 216	6 400

Анализ производительности с помощью утилиты psi.

Ядро oddEvenShared – глобальные обращения выполнены аккуратно, а локальная память не используется.

Метрика	Значение (суммарно)	Комментарий
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum	262 144 сектора	Соответствует полному объёму блока (1024 × 256 байт), лишних загрузок нет
l1tex__t_sectors_pipe_lsu_mem_global_op_st.sum	262 144 сектора	Каждый элемент записан ровно один раз
sm__sass_inst_executed_op_local.sum	0	Отсутствие обращений к локальной памяти (регистровый спилл отсутствует)

Ядро bitonicGlobal (работа напрямую с глобальной памятью) – отсутствие конфликтов разделяемой памяти и локальной памяти, предсказуемый трафик.

Метрика	Значение (суммарно)	Комментарий
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum	0	Разделяемая память не используется, конфликтов нет
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum	0	Аналогично
sm__sass_inst_executed_op_local.sum	0	Нет спиллов
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum	262 144 сектора	Каждый вызов ядра загружает ровно блок данных (8192 × 256 / 32 = 262144 сектора)

Метрика	Значение (суммарно)	Комментарий
l1tex__t_sectors_pipe_lsu_mem_global_op_st.sum	262 144 сектора	Сохранение того же объёма

Ядро bitonicGlobalShared – наилучшие показатели: нулевые конфликты банков и полное отсутствие расходящихся ветвлений в большинстве запусков.

Метрика	Значение (суммарно)	Комментарий
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum	0	Идеальное выравнивание доступа, банковские конфликты отсутствуют
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum	0	Аналогично для записи
sm__sass_inst_executed_op_local.sum	0	Регистровой памяти достаточно, спиллов нет
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum	262 144 сектора	Каждый блок загружается полностью, без избыточности
l1tex__t_sectors_pipe_lsu_mem_global_op_st.sum	262 144 сектора	Сохранение выполнено эффективно
smssp__branch_targets_threads_divergent	65 536 в некоторых запусках	Расходящиеся ветвления присутствуют только в части вызовов, в остальных – 0, что указывает на

Метрика	Значение (суммарно)	Комментарий
		равномерную нагрузку.

Общая эффективность. Во всех ядрах отсутствует локальная память, обращения к глобальной памяти строго соответствуют размерам обрабатываемых данных, а в ядрах с разделяемой памятью (bitonicGlobalShared) достигнуто бесконфликтное использование банков. Секторные метрики стабильны и предсказуемы, что свидетельствует о корректной настройке сетки блоков и потоков.

Выводы

Реализованный алгоритм блочной сортировки чёт-нечёт с последующим битоническим слиянием предназначен для эффективной сортировки больших одномерных массивов целых чисел на графических процессорах NVIDIA. Область применения охватывает задачи, требующие высокой пропускной способности: обработка потоковых данных в реальном времени (например, сортировка результатов запросов в базах данных, предобработка больших объёмов сейсмических или финансовых данных), а также ускорение этапов сортировки в составе сложных гибридных алгоритмов (например, в комбинации с фильтрацией или слиянием). Алгоритм особенно эффективен, когда размер массива достигает миллионов элементов, где накладные расходы на запуск ядер окупаются массовым параллелизмом.

Сложность программирования оказалась довольно высокой. Основные трудности были связаны с обеспечением бесконфликтного доступа к разделяемой памяти (банковские конфликты в ядре oddEvenShared потребовали тщательного выбора размера блока и порядка обменов) и корректной синхронизацией между этапами битонического слияния. Также пришлось аккуратно обрабатывать граничные условия (дополнение массива до степени двойки значением INF) и выбирать порог для переключения между ядрами, работающими через глобальную память (bitonicGlobal) и через разделяемую (bitonicGlobalShared). Проблемой стало и расходящееся ветвление внутри варпов на поздних стадиях слияния, что было частично смягчено использованием разделяемой памяти и фиксированным размером блоков.

Полученные результаты профилирования показывают, что для массива из 8 388 608 элементов (2^{23}) время полной сортировки на GPU составляет около 68 мс, что в 45 раз быстрее однопоточной сортировки на CPU (≈ 3100 мс). На малых размерах (до 4096 элементов) CPU ожидаемо опережает GPU из-за накладных расходов на передачу данных и запуск ядер. Оптимальная конфигурация – 1024 потока в блоке для предварительной сортировки и 512 потоков для битонического слияния – позволяет полностью загрузить все 40 мультипроцессоров GPU и достичь нулевых банковских конфликтов в ядре bitonicGlobalShared. Метрики `l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum` и `sm__sass_inst_executed_op_local.sum`, равные нулю, подтверждают отсутствие лишних обращений к глобальной

памяти и регистрового спилла. Таким образом, реализованная блочная сортировка является сбалансированным решением, пригодным для ускорения реальных приложений с большими объёмами данных.